

Criterion C: Documentations

Product Structure

The structure of this project follows object-oriented programming (OOP) for computations with Java Swing GUI and MySQL database to accomplish all of the necessary requirements as outlined in the success criteria. It achieves Model-View-Controller (MVC) design pattern by separating DB access, GUI display and computations separately. The JavaDBAccess and InstallDB classes help execute queries with MySQL, GUI classes each handle different information from Admin Controls to Performance Evaluations and more. The controller classes help with high-level computations such as employee searching, bonus calculations, user role segmentation, and pie chart rendering separate from GUI.

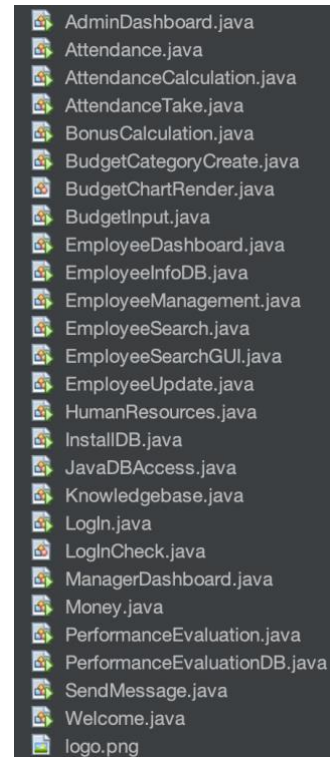


Figure 1. List of GUI classes.

Algorithms and Features

Log In Handling

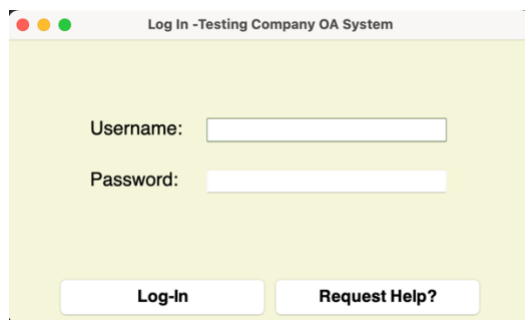


Figure 2A. GUI for Log In screen.

Log-In is an integral part of the code that allows user to gain access to the software itself while implementing user access control where each personnel is only able to access relevant functionalities inside of the program. The algorithm allows the user's entry to be checked with Users Table in the database and assign role-specific access redirections.

```

if (source == logInButton)
{
    this.username = usernameField.getText();
    this.password = String.valueOf(passwordField.getPassword());
    this.dbAccess = new JavaDBAccess("OfficeAutomationDB");

    boolean userExist = false;
    userExist = LogInCheck.checkLogIn(username, password, dbAccess);

    if (userExist)
    {
        if (role.equals("Admin"))
        {
            new AdminDashboard();
            this.dispose();
        }
        else if (role.equals("Manager"))
        {
            new ManagerDashboard();
            this.dispose();
        }
        else if (role.equals("Employee"))
        {
            new EmployeeDashboard();
            this.dispose();
        }
    }
}

```

Figure 2B. Code snippet on user access control.

```

public static boolean checkLogIn(String username, String password, JavaDBAccess dbAccess)
{
    Object[][] data = dbAccess.getUsersData();
    boolean match = false;
    int r = 0;
    String dbUsername = "";
    String dbPassword = "";

    while (r < data.length && match == false)
    {
        dbUsername = data[r][0].toString();
        dbPassword = data[r][1].toString();
        if (dbUsername.equals(username) && dbPassword.equals(password))
        {
            LogIn.setRole(data[r][2].toString());
            match = true;
        }
        r++;
    }
    return match;
}

```

Figure 2C. Algorithm (#1) code snippet that checks database for match with username and password and sets user with the corresponding role after finding a match.

Pie Chart Render

Pie chart is used to show budget information in that consists of the spending categories and the amount allocated. It is rendered using an external library, JFreeChart, with data provided from the budget table into the GUI.

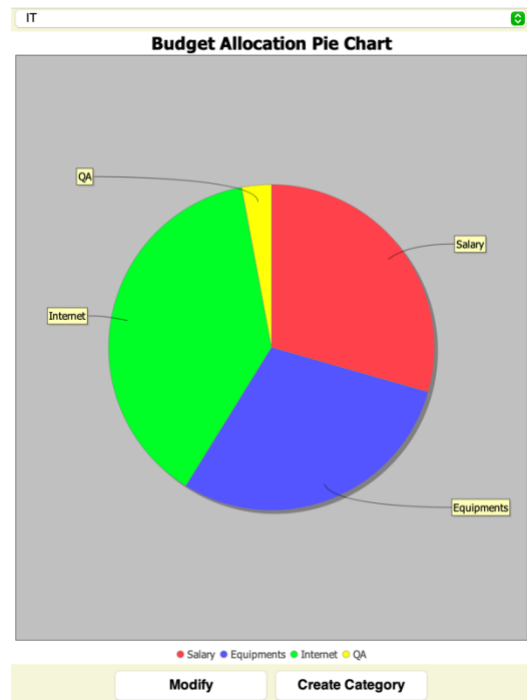


Figure 3A. Pie Chart for IT department budget.

```
else if (e.getSource() == departmentDropdown)
{
    int selectedIndex = departmentDropdown.getSelectedIndex();
    if (selectedIndex >= 0)
    {
        int selectedDeptID = departmentIDs[selectedIndex]; // Get department ID
        chartRender = new BudgetChartRender(selectedDeptID); // Create new chart
        leftPanel.removeAll(); // Removes everything
        leftPanel.add(departmentDropdown, BorderLayout.NORTH); // Re-add dropdown
        leftPanel.add(chartRender, BorderLayout.CENTER);
        leftPanel.add(buttonOnLeftPanel, BorderLayout.SOUTH);
        leftPanel.revalidate();
        leftPanel.repaint();
    }
}
```

Figure 3B. Code snippet in GUI that helps display chart in the GUI class with a department is selected in the dropdown. (Code defaults to first department in the database table if none is manually selected)

```

public void updateChart()
{
    DefaultPieDataset dataset = new DefaultPieDataset();

    // Retrieve budget data for the selected department
    Object[][] budgetData = dbAccess.getBudgetData();

    for (int r = 0; r < budgetData.length; r++)
    {
        int deptID = Integer.parseInt(budgetData[r][1].toString()); // Department ID column
        if (deptID == departmentID)
        {
            // Retrieves Budget Category
            String category = budgetData[r][2].toString();
            // Retrieves Allocated Amount
            double allocatedAmount = Double.parseDouble(budgetData[r][3].toString());
            // Sets value of piechart by category using allocated amount vs total allocated
            dataset.setValue(category, allocatedAmount);
        }
    }

    // Generate Pie Chart
    JFreeChart chart = ChartFactory.createPieChart(
        "Budget Allocation Pie Chart", // Chart Title
        dataset, // Pie chart data source
        true, // Shows legend
        true, // Info shown when hovered (tooltips)
        false // Disables chart clicking & modification (URLS)
    );

    ChartPanel chartPanel = new ChartPanel(chart);
    removeAll(); // Clear previous chart to ensure update everytime
    this.add(chartPanel, BorderLayout.CENTER);
    this.revalidate();
    this.repaint();
}

```

Figure 3C. Algorithm (#3) code snippet of how pie-chart is generated.

Messages Display

Messages are sent by Admins/Managers in their respective dashboard's function bar. The messages can either be sent as regular or for knowledge base in the SendMessage class. The content will then be posted to corresponding dashboards and refresh.

Message Table			
Sender	Title	Message Content	Sent Date
Admin	Testing	Message DB is working!	2025-02-02
admin	SendMessage UI Testing	Did it work? If you see this, then it worked!	2025-02-03
Admin	testfinal	FINAL TeSt	2025-02-11
manager	manTest	OVERRR	2025-02-11

Figure 4B. A list of messages that is posted into the database which shows on the dashboard of every level.

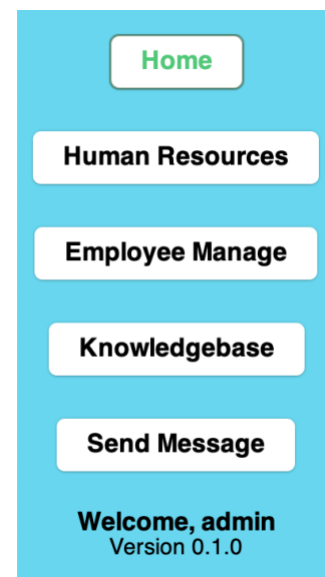


Figure 4A. Admin Dashboard Function Bar for navigation.

```

if (e.getSource() == postButton)
{
    // Initialize insert values
    String title = titleField.getText().trim(); // .trim to clear white spaces in front/after
    String content = contentArea.getText().trim();
    String username = LogIn.getUsername();
    String date = LocalDate.now().toString(); // To retrieve current date (reduce user input error)
    String category = (String) categoryDropdown.getSelectedItemAt(); // Receives whether to add to Messages OR KB

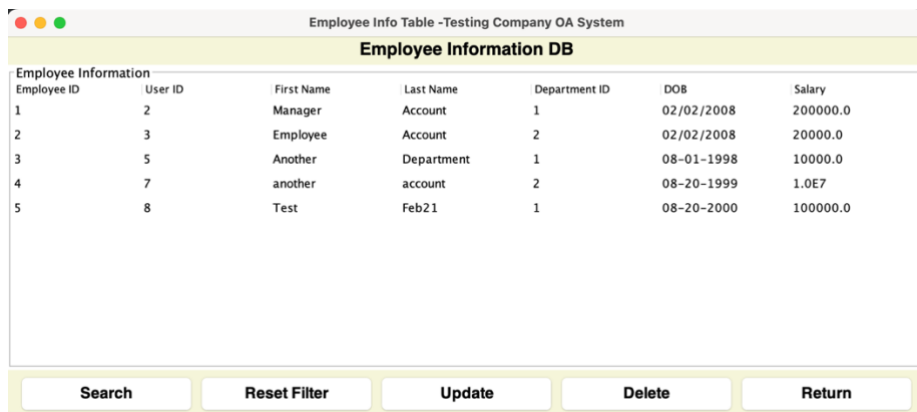
    if (title.isEmpty() || content.isEmpty())
    {
        JOptionPane.showMessageDialog(this, "Title and Content cannot be empty.", "Error Empty Field", JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (category.equals("Message"))
    {
        dbAccess.insertMessage(username, title, content, date); // Insert into messageDB
    }
    else if (category.equals("Knowledge Base"))
    {
        dbAccess.insertKB(username, title, content, date); // Insert into knowledgebaseDB
    }
    // Clears field after insertion
    titleField.setText("");
    contentArea.setText("");
    JOptionPane.showMessageDialog(this, "Content posted successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
    if (LogIn.getRole().equals("Admin"))
    {
        AdminDashboard.updateTable();
    }
    else if (LogIn.getRole().equals("Manager"))
    {
        ManagerDashboard.updateTable();
    }
}
}

```

Figure 4C. Code snippet that shows how messages are added into the database after the user posts a message and how dashboard tables refresh after.

Employee Management

Employees are all currently stored in a database table and displayed through a GUI frame which contains their first and last name, date of birth, department, and their salary. User with access can create, update, and delete any entries as needed. A search function is also available where the user can find an employee by name. Additional robustness checks for exceptions are also implemented throughout the algorithm to ensure that the modifications are done successfully.



Employee Information DB						
Employee ID	User ID	First Name	Last Name	Department ID	DOB	Salary
1	2	Manager	Account	1	02/02/2008	200000.0
2	3	Employee	Account	2	02/02/2008	20000.0
3	5	Another	Department	1	08-01-1998	10000.0
4	7	another	account	2	08-20-1999	1.0E7
5	8	Test	Feb21	1	08-20-2000	100000.0

Search Reset Filter Update Delete Return

Figure 5A. Employee Database Table GUI frame accessed through JavaDBAccess

```

public static Object[][] searchEmployees(String searchTerm)
{
    JavaDBAccess dbAccess = new JavaDBAccess("OfficeAutomationDB");
    Object[][] allEmployees = dbAccess.getEmployeeData();

    String allLower = searchTerm.toLowerCase().trim(); // Standardize search and remove space in front and after

    // Filter Employees Matching Search Term (Search by name)
    return java.util.Arrays.stream(allEmployees)
        .filter(row -> {
            String firstName = row[2].toString().toLowerCase();
            String lastName = row[3].toString().toLowerCase();
            String fullName = firstName + " " + lastName;

            return firstName.contains(allLower) || // Match first name
                lastName.contains(allLower) || // Match last name
                fullName.contains(allLower); // Match full name
        })
        .toArray(Object[][]::new);
}

```

Figure 5B. Algorithm (#4) code Snippet of the search function by adding filter to the employee database table based on the search term inputted by the user

```

// Main Panel Buttons
if (source == createAccountButton)
{
    String username = JOptionPane.showInputDialog(this, "Enter Username:");
    String password = JOptionPane.showInputDialog(this, "Enter Password:");
    String[] roles =
    {
        "Employee", "Manager", "Admin"
    };
    String role = (String) JOptionPane.showInputDialog(this, "Select Role:",
        "Role Selection", JOptionPane.QUESTION_MESSAGE, null, roles, roles[0]);

    if (username != null && password != null && role != null)
    {
        int userID = dbAccess.insertUsersData(username, password, role); // Returns userID
        if (userID != 0) // Starts at 1 so if not 0 meaning created correctly.
        {
            // If employee account created, auto jump to employee creation to avoid missing DB Entries
            if (role.equals("Employee") || role.equals("Manager"))
            {
                addEmployee(userID); // pre-defined method to add employee to DB
            }
            JOptionPane.showMessageDialog(this, "Account successfully created with User ID: " + userID);
        }
        else
        {
            JOptionPane.showMessageDialog(this, "Error creating account.", "Error Account Creation",
                JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

Figure 5C. Code snippet of the account creation process. Robustness checks are done to check for incorrect user ID generation and allows for users to be created successfully

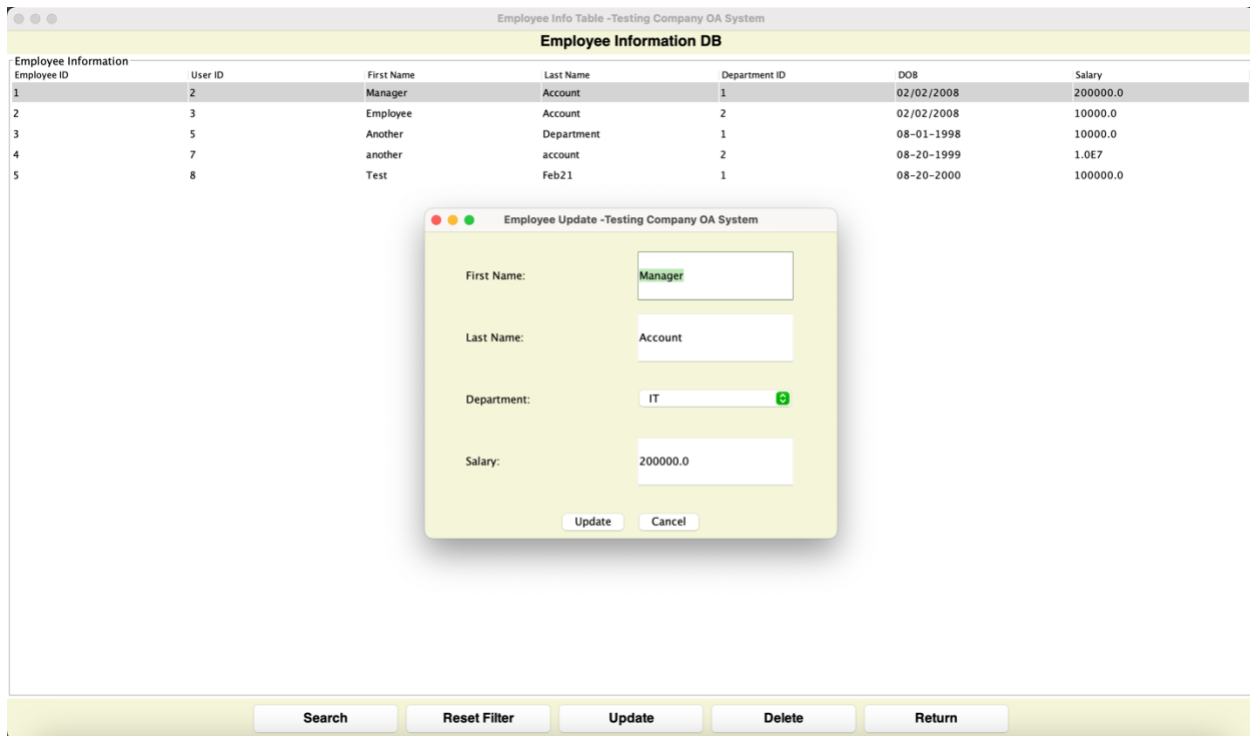


Figure 5D. GUI of the pop-up screen to update an employee after a user is selected in the database table

```

if (e.getSource() == updateButton)
{
    try
    {
        String firstName = firstNameField.getText().trim();
        String lastName = lastNameField.getText().trim();
        double salary = Double.parseDouble(salaryField.getText().trim());
        int departmentID = departmentIDs[deptDropdown.getSelectedIndex()];

        if (firstName.isEmpty() || lastName.isEmpty() || salary < 0)
        {
            JOptionPane.showMessageDialog(this, "Invalid input!", "Error Invalid Input",
                JOptionPane.ERROR_MESSAGE);
            return;
        }

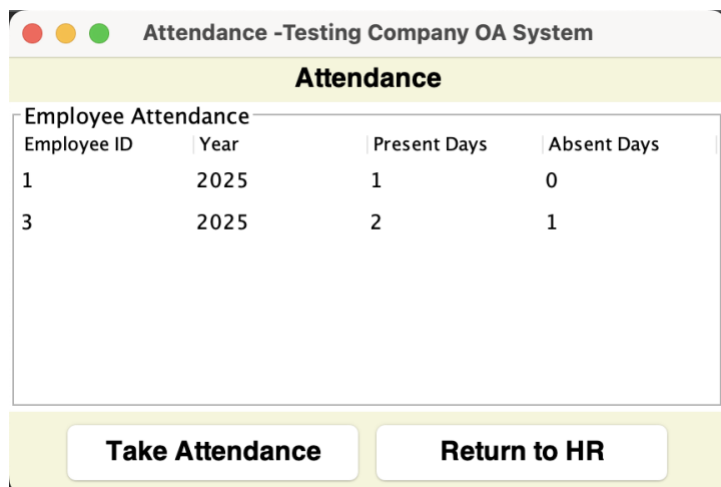
        dbAccess.updateEmployee(employeeID, firstName, lastName, departmentID, salary);
        JOptionPane.showMessageDialog(this, "Employee updated successfully!", "Success",
            JOptionPane.INFORMATION_MESSAGE);
        this.dispose();
        EmployeeInfoDB.refreshTable(dbAccess.getEmployeeData()); // Refresh Table
    }
    catch (NumberFormatException ex)
    {
        JOptionPane.showMessageDialog(this, "Invalid salary input!", "Error Invalid Number",
            JOptionPane.ERROR_MESSAGE);
    }
}

```

Figure 5E. Code snippet of how employees are updated in the database after "Update" button is clicked. The update MySQL query is done with updateEmployee() method in JavaDBAccess.java; robustness checks are performed to check for invalid numbers format such as letters or special symbols.

Attendance Taking

The attendance record is shown in a table with their name and department ID and the user has the option to take attendance for the employee in a separate pop-up screen where an attendance year needs to be entered in addition to their attendance status to track the information over the year. Once submitted, a separate computation is done to ensure that the right category (Absent/Present) gets incremented.



Employee ID	Year	Present Days	Absent Days
1	2025	1	0
3	2025	2	1

Figure 6A. Attendance Database table with employee ID, attendance year, and their days of present/absent.

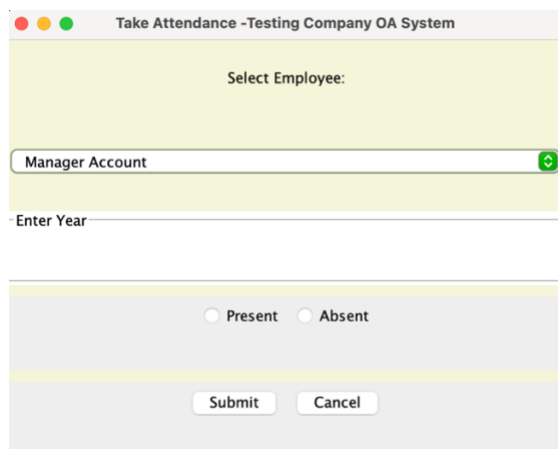


Figure 6B. The attendance taking screen where user with permission can add an attendance record to the employee.

```
public static void updateAttendance(int employeeID, String year, boolean isPresent)
{
    JavaDBAccess dbAccess = new JavaDBAccess();
    dbAccess = new JavaDBAccess("OfficeAutomationDB"); // Instantiate JavaDbAccess (used for later insert/update)
    int presentIncrement = isPresent ? 1 : 0; /* If else */
    int absentIncrement = isPresent ? 0 : 1;
    dbAccess.updateAttendance(employeeID, year, presentIncrement, absentIncrement);
}
```

Figure 6C. Algorithm (#5) code snippet to increment the corresponding attendance count by 1 based on the user input in Figure 6B.

Technique Usage

1. Database Access

Database in the program is accessed by a specific class called JavaDBAccess.java within the package that has various methods to retrieve relevant and necessary information to allow the flow of the program without directly integrating SQL code within GUI and computation classes to enhance the MCV and OOP theme.

2. Object-Oriented Programming (OOP)

This program utilizes a lot of the fundamentals of OOP by creating encapsulated variables (Figure 7A) that are modifiable and/or accessible only via specific accessor methods (Figure 7B). It provides a modular design by separating portions of the software for different functions and responsibilities and maximizes the code reusability and ease of maintenance.

```
// Declaring necessary components for Frame
private JPanel mainPanel;
private JPanel buttonPanel;

private JLabel usernameLabel;
private JLabel passwordLabel;
private JTextField usernameField;
private JPasswordField passwordField;

private JButton loginButton;
private JButton requestHelpButton;

private JavaDBAccess dbAccess;
private static String username;
private static String role = "";
private String password;
```

Figure 7A. Example of private variables and encapsulation within the LogIn.java class

```
/* Encapsulation Accessor (For use across application) */
public static void setUsername(String u)
{
    username = u;
}

public static String getUsername()
{
    return username;
}

public static void setRole(String r)
{
    role = r;
}

public static String getRole()
{
    return role;
}
```

Figure 7B. Example of accessor methods for encapsulated variables within LogIn.java class

3. Data Structure

Data structures are utilized in this program to fill the various database tables along with other use cases that may require it such as pie-chart or salary retrieval in many cases. Data Structure consists mostly of 2D arrays that utilizes a JavaDBAccess class variable and methods to retrieve relevant information from the database.

```
// Constructing Messages Table with DB
this.dbAccess = new JavaDBAccess("OfficeAutomationDB");
Object[][] data = dbAccess.getMessagesData();
```

Figure 8A. Use of data structure (2D Array) to retrieve messages from messages database table

```
// Convert datalist from table into 2D array for access for class
public Object[][] to2dArray(ArrayList<ArrayList<String>> dataList)
{
    if (dataList.isEmpty()) // if table is empty
    {
        Object[][] dataArray = new Object[0][0];
        return dataArray;
    }
    else
    {
        // Number of columns
        int columnCount = dataList.get(0).size();
        Object[][] dataArray = new Object[dataList.size()][columnCount];
        // Read each cell of each row into array
        for (int r = 0; r < dataList.size(); r++)
        {
            ArrayList<String> row = dataList.get(r); // Get the row
            for (int c = 0; c < columnCount; c++)
            {
                dataArray[r][c] = row.get(c); // Get the cell
            }
        }
        return dataArray;
    }
}
```

Figure 8B. to2dArray method in JavaDBAccess.java class to convert any given table and column into a 2D Array return

```
/* Messages Table */
public Object[][] getMessagesData()
{
    String[] columnNames =
    {
        "username", "messageTitle", "messageContent", "sentDate"
    };
    return to2dArray(getData("messagesTable", columnNames));
}
```

Figure 8C. Method used in Figure 8A. to retrieve messages from database as a 2D Array

4. Dynamic GUI

On the left-hand side of Money.java class for employee salary and bonus, when an employee's name is selected, their salary will correspondingly update as well. In addition, when clicking "Bonus Calculation" button below, the bonus calculation will be calculated in accordance to the selected employee's salary.

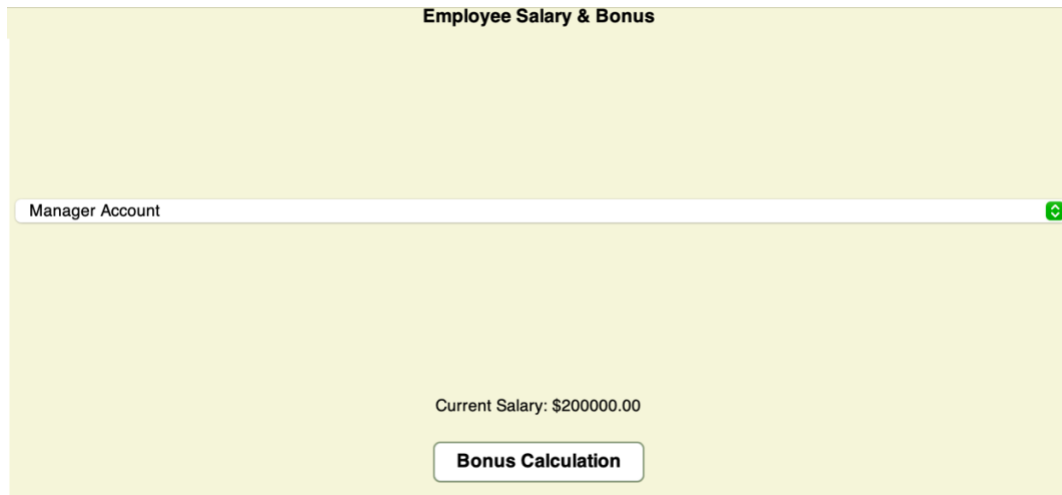


Figure 9A. Left side of Money.java GUI with first employee selected and their first and last name (Name in example is used for testing purposes only. Real-life usage will consist of personal information)

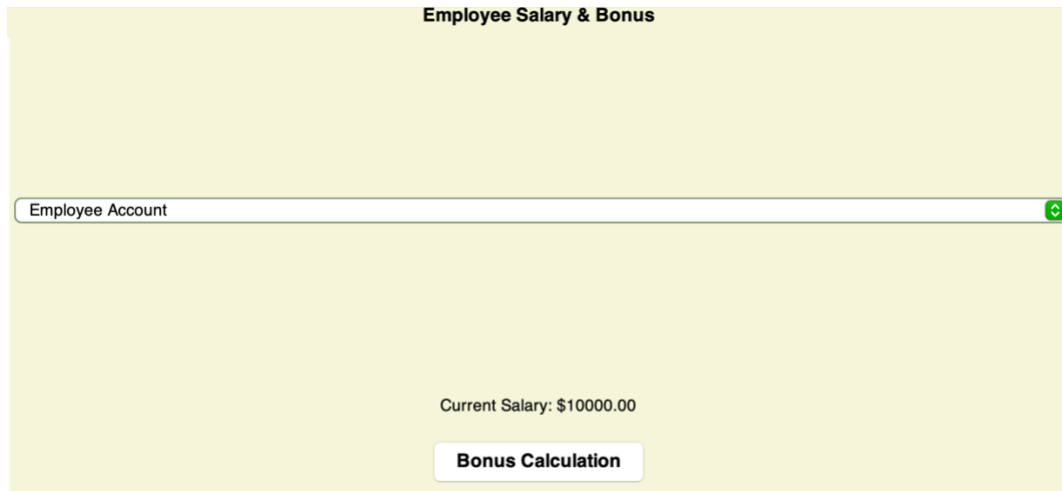


Figure 9B. Left side of Money.java GUI with another employee selected different from figure 9A.

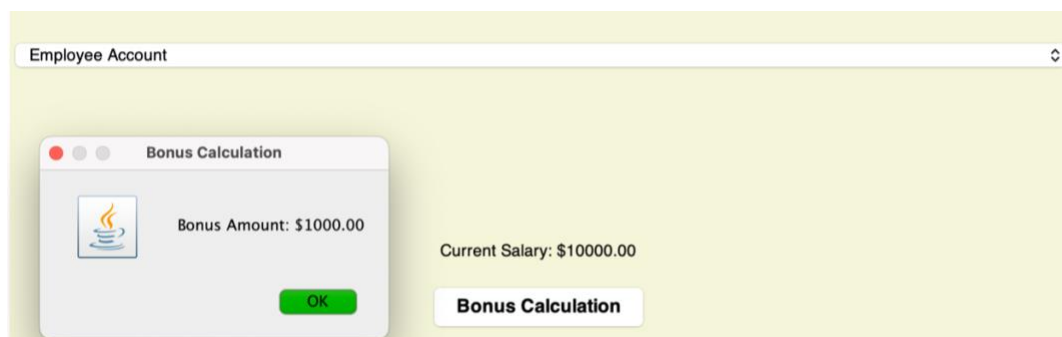


Figure 9C. Calculated Bonus of the selected employee and user input of 10% bonus rate

```

else if (e.getSource() == employeeDropdown)
{
    int employeeIndex = employeeDropdown.getSelectedIndex();
    // Retrieve and update salary based on selected employee
    if (employeeIndex >= 0)
    {
        // Retrieves from column 6 in employeesDB (Salary)
        double salary = Double.parseDouble(employeeData[employeeIndex][6].toString());
        // converts to 2 decimal place for money
        String formattedSalary = String.format("%.2f", salary);
        salaryLabel.setText("Current Salary: $" + formattedSalary);
        // Text Update after new selection
        salaryLabel.revalidate();
        salaryLabel.repaint();
    }
}
}

```

Figure 9D. Code snippet of how the salary numbers are dynamically updated

5. ActionListener and Event Handling

ActionListener is used across all GUI classes that requires useable buttons, dropdowns, and more. It allows a trigger based on user actions such as clicking or selecting an object and allows for a responsive and interactive user experience. The trigger is assigned in the method actionPerformed.

```

if (source == homeButton)
{
    // Current Page
}
else if (source == hrButton)
{
    new HumanResources();
    this.dispose();
}
else if (source == employeeButton)
{
    new EmployeeManagement();
    this.dispose();
}
else if (source == knowledgeButton)
{
    new Knowledgebase();
    this.dispose();
}
else if (source == sendMessageButton)
{
    new SendMessage();
}
else if (source == logoutButton)
{
    new Welcome(); // Return back to Welcome Page
    this.dispose();
}
}

```

Figure 10. Code snippet of a series of buttons used to construct function bar (Figure 4A.) for dashboards with trigger action in the actionPerformed method

6. Static Variables/Methods

Static Variables and methods in the code is used for global access and maintaining data consistency across classes. In addition, it is also used to eliminate creation of object to access which improves operation efficiency of the code.

```
public static final Color BG_COLOR = new Color(245, 245, 220); // Beige
public static final Color FONT_COLOR = new Color(0, 0, 0); // black
public static final Color FOOTER_COLOR = new Color(0, 0, 0); // black
public static final Color WARNING_COLOR = new Color(255, 0, 0); // red
public static final Font THEME_TITLE_FONT = new Font("Helvetica", Font.BOLD, 30);
public static final Font THEME_DESCRIPTION_FONT = new Font("Helvetica", Font.PLAIN, 18);
```

Figure 11A. Code snippet of static variables in order to reuse in other classes.

```
// Structuring the frame
this.setSize(500, 300);
this.setLocationRelativeTo(null);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
this.getContentPane().setBackground(Welcome.BG_COLOR);
this.setLayout(new BorderLayout());

// Constructing main panel
mainPanel = new JPanel();
mainPanel.setBackground(Welcome.BG_COLOR);
mainPanel.setLayout(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();
```

Figure 11B. Two instances of the static variable "BG_COLOR" being used in another class (Login.java).

7. Iteration/Loops

Loops are used throughout the program to handle and process potentially larger datasets such as information from database tables and allows for scalability. The program primarily uses for-loops and at times while-loops retrieve, display, and modify data accordingly.

```
// DB data for all Performance Evaluations
for (int r = 0; r < data.length; r++)
{
    tableModel.addRow(data[r]); // adds whole row for table
}
```

Figure 12A. Code snippet of for-loop being used in PerformanceEvaluationDB.java to display the information in a table.

```

while (r < data.length && match == false)
{
    dbUsername = data[r][0].toString();
    dbPassword = data[r][1].toString();
    if (dbUsername.equals(username) && dbPassword.equals(password))
    {
        LogIn.setRole(data[r][2].toString());
        match = true;
    }
    r++;
}

```

Figure 12B. Code snippet of while-loop being used in LogInCheck.java that iterates through user's database table and finds match for corresponding username and password entered by the user

8. Input Validations

Input validations throughout the program was used extensively in order to ensure that the user errors are minimized and data format and integrity is protected – in an effort to stop SQL injections. Fields in the inputs from the user is checked to ensure that only related characters and formats are allowed such as names only allowing letters and date of birth is checked for unrealistic ages such as negative or ages over 130 years old.

```

while (nameEmpty)
{
    firstName = JOptionPane.showInputDialog(this, "Enter Employee's First Name: ");
    lastName = JOptionPane.showInputDialog(this, "Enter Employee's Last Name: ");
    if (firstName != null && lastName != null && !firstName.trim().isEmpty() && !lastName.trim().isEmpty()
        && firstName.matches("[a-zA-Z]+") && lastName.matches("[a-zA-Z]+"))
    {
        nameEmpty = false;
    }
    else
    {
        JOptionPane.showMessageDialog(this, "First/Last Name cannot be empty and must contain only letters!", "Error Invalid Name",
            JOptionPane.ERROR_MESSAGE);
    }
}

```

Figure 13A. The employee's name can only contain letters and cannot be empty.

9. Java LocalDate retrieval

The Java LocalDate class was used in order to validate the user's input and other date-related queries in the Office Automation System. In terms of input validation, the account creation relies heavily on the correct format for the Employee's DOB in order to check for age constraints, future dates and data cleanliness in the database. In addition, it is also used when making a post (messages/knowledgebase articles/performance evaluations) in order to retrieve the date of publication from the author.


```
String dob = "";
boolean validDOB = false;
while (!validDOB)
{
    dob = JOptionPane.showInputDialog(this, "Enter Employee's Date of Birth (YYYY-MM-DD): ");
    try
    {
        java.time.LocalDate birthDate = java.time.LocalDate.parse(dob);
        java.time.LocalDate today = java.time.LocalDate.now();
        int age = java.time.Period.between(birthDate, today).getYears();

        if (birthDate.isAfter(today))
        {
            JOptionPane.showMessageDialog(this, "Date of birth cannot be in the future!", "Error Invalid DOB", JOptionPane.ERROR_MESSAGE);
        }
        else if (age < 0 || age > 130)
        {
            JOptionPane.showMessageDialog(this, "Employee must be of a real age.", "Error Invalid Age", JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            validDOB = true;
        }
    }
    catch (Exception ex)
    {
        JOptionPane.showMessageDialog(this, "Invalid date format! Please use YYYY-MM-DD.", "Error Invalid Format", JOptionPane.ERROR_MESSAGE);
    }
}
}
```

Figure 14A. Java LocalDate Function being used to check for Date of Birth and future dates.

```
// Initialize insert values
String title = titleField.getText().trim(); // .trim to clear white spaces in front/after
String content = contentArea.getText().trim();
String username = LogIn.getUsername();
String date = LocalDate.now().toString(); // To retrieve current date (reduce user input error)
String category = (String) categoryDropdown.getSelectedItem(); // Receives whether to add to Messages OR KB
```

Figure 14B. Java LocalDate function being used to retrieve author's current date for date of publication.

10. Iterative Design

Iterative Design was used thoroughly in this product and adopted well before the actual development of the product. Multiple parties were actively involved in the development of this product including the instructor and the clients, as outlined in the appendix. The clients are mostly involved with the functionalities that the product presented and gave criteria on a successful product while the instructor mainly focused on robustness of the program. The product was adapted and amended each time with feedback and revised to enhance functionality, user experience, and system security.

External Tools and Libraries

JDBC Connector

Java Database Connectivity Connector is used to bridge Java and MySQL database together which allows queries to be directly executed through Java.

JFreeChart

JFreeChart is an open-source public library for generating dynamic charts. In the program, it is used to generate pie-charts to visualize budget allocations.

Apache Netbeans (IDE)

Apache NetBeans is used as the IDE for the entire project which provided help and support in various aspects such as refactoring variable, method, and class names, error detection, and code suggestions.

Word Count: 1082

References

[1] Oracle Corporation, Java Platform, Standard Edition Documentation. Available:

<https://docs.oracle.com/en/java/javase/>

[2] MySQL, MySQL Connector/J 8.0 Developer Guide. Available:

<https://dev.mysql.com/doc/connector-j/en/>

[3] JFreeChart Project, JFreeChart – Java Chart Library. Available:

<https://sourceforge.net/projects/jfreechart/>

[4] Apache Software Foundation, NetBeans IDE Documentation. Available:

<https://netbeans.apache.org/kb/docs/java/gui-functionality.html>

[5] A. Chang, JavaDBAccess.java – Database Access and Modification Class (Modified from 2023 version), Intro to Computer Science Project, 2025